

2026-08-07 “CONST-033 poweroff-signal triage (host_no_auto_poweroff_challenge.sh test 4/4 FAIL)

Revision: 1 Last modified: 2026-08-07T21:16:00Z

TL;DR

challenges/scripts/host_no_auto_poweroff_challenge.sh flagged 1/5 checks failing: one "The system will power off now!" journal broadcast recorded after the documented CONST-033 fix marker timestamp (2026-04-28T20:54:31+03:00). Systematic read-only triage (per the CLAUDE.md “CONST-033 Operational Note”[◆]) traced the broadcast to a **single, real, orderly, human-initiated shutdown+reboot cycle on 2026-08-03 13:36:25–13:37:26 MSK** “confirmed by helix-redis™’s own log line "User requested shutdown..." plus a completely clean systemd/GNOME shutdown teardown sequence (no crash, no panic, no forced power loss).

This is NOT a suspend, NOT an OOM-triggered freeze, and NOT caused by any automation, script, or AI agent in this project. It is also **not** one of the previously-documented “common false positive”[◆] classes (screen lock, compositor stall, foreign container OOM) “it is a genuine, one-time, deliberate host power-off/reboot performed by the human operator, five days before this triage, that the CONST-033 hard ban (which targets *automated/idle-triggered* power transitions, not manual operator action) was never designed to prevent. The host has been running continuously since the resulting boot (4+ days at time of triage) with zero further suspend/poweroff/OOM events.

Triage sequence (read-only, per the documented procedure)

1. uptime

```
$ uptime
00:09:51 up 4 days, 10:32, 3 users, load average: 1.16, 0.72, 0.67
```

4 days 10h32m of continuous uptime as of the start of triage. Working backwards, the current boot started around **2026-08-03 ~13:37** “i.e. the uptime figure itself is consistent with (and points directly at) the exact event under investigation, rather than ruling it out. This escalated the triage from “likely false positive”[◆] to “confirm what actually happened.”[◆]

2. Re-ran the challenge script for exact current output

Confirmed genuinely read-only first (only `systemctl is-enabled`, `grep` of config files, and `journalctl` reads `â€”` no mutating commands anywhere in the script). Then ran it:

```
=== host_no_auto_poweroff_challenge ===
[1/4] sleep / suspend / hibernate / hybrid-sleep targets masked?
    sleep.target: masked
    suspend.target: masked
    hibernate.target: masked
    hybrid-sleep.target: masked
PASS: all 4 sleep targets masked
[2/4] AllowSuspend=no in sleep.conf or drop-in?
PASS: AllowSuspend=no present
[3/4] logind IdleAction and HandlePowerKey safe?
    logind IdleAction: ignore
    logind HandlePowerKey: ignore
PASS: IdleAction=ignore (safe)
PASS: HandlePowerKey=ignore (safe)
[4/4] journal: any 'will power off' broadcast since fix?
    fix applied at: 2026-04-28T20:54:31+03:00
    'will suspend/power off' broadcasts since fix: 1
FAIL: 1 suspend/poweroff events since fix â€” guarding didn't take

=== summary: 4 pass, 1 fail ===
```

4 of 5 assertions PASS (all sleep/hibernate/hybrid-sleep targets masked, AllowSuspend=no present, IdleAction=ignore, HandlePowerKey=ignore). Only the `â€œzero poweroff broadcasts since fixâ€` assertion FAILs, on exactly one event.

3. Kernel-log check for actual suspend invocation

```
$ journalctl -k --since "2026-04-28" | grep -iE "will suspend|systemd-suspend"
(0 matches)
```

Zero matches across the entire window since the fix marker. **systemd never invoked suspend at any point.** This rules out a suspend/resume cycle entirely `â€”` whatever happened, it was not a suspend.

4. Kernel-log check for OOM-kills

```
$ journalctl -k --since "2026-04-28" | grep -iE "oom-kill|killed process"
(0 matches)
```

Zero matches across the entire window. No OOM-kill `â€”` of any container, any user slice, anything `â€”` occurred anywhere in this window, including around the flagged event. This rules out the `â€œOOM misperceived as freeze/suspendâ€` false-positive class documented in the prior 2026-04-27 incident.

5. Source-tree scanner (no_suspend_calls_challenge.sh)

```
=== no_suspend_calls_challenge ===  
FAIL: forbidden host-power-management invocations (CONST-033):  
constitution/docs/scripts/guard-forbidden-commands.md:109:Blocks `systemct  
constitution/docs/scripts/guard-forbidden-commands.md:110:`loginctl suspen  
constitution/docs/scripts/guard-forbidden-commands.md:111:`pm-suspend|pm-h  
=== summary: FAIL ===
```

Separate, pre-existing, unrelated finding â€” this is a scanner false-positive, not a real invocation. The three flagged lines are inside `constitution/docs/scripts/guard-forbidden-commands.md`, which is *documentation describing the PreToolUse guard hook*â€™s own denylist (i.e. prose that quotes the forbidden command names so operators know what the hook blocks). The scannerâ€™s `EXCLUDE_PATHS` allowlist already exempts several governance docs by design (`CONSTITUTION.md`, `AGENTS.md`, `HOST_POWER_MANAGEMENT.md`, etc.) but does not yet include this specific doc path. No actual forbidden command is invoked anywhere in the source tree. **Not modified as part of this triage** (read-only mandate) â€” flagged here for a future fix-forward commit to extend `EXCLUDE_PATHS` in `scripts/host-power-management/check-no-suspend-calls.sh`.

6. Root-cause identification: what actually happened at the flagged timestamp

`journalctl --list-boots` shows the exact boot boundary:

IDX	BOOT	ID	FIRST ENTRY	LAST ENTRY
-1	c3f995c223e74687b0fc7a1f187c2486	Sun 2026-08-02 10:41:24 MSK	Mon 2026-	
0	f32e9f891c8043bc90e716518e9c6851	Mon 2026-08-03 13:37:26 MSK	Sat 2026-	

A real boot boundary: the previous bootâ€™s last journal entry is 2026-08-03 13:36:32, the current bootâ€™s first entry is 2026-08-03 13:37:26 â€” a ~54-second gap consistent with an orderly shutdown-then-power-on cycle (not a crash/panic, which would show an abrupt truncation with no clean shutdown sequence).

The flagged broadcast and its immediate context:

```
2026-08-03T13:36:25+03:00 nezha systemd-logind[1263]: The system will powe  
2026-08-03T13:36:25+03:00 nezha systemd-logind[1263]: System is powering d  
2026-08-03T13:36:25+03:00 nezha systemd[1]: Stopping session-25.scope - Se  
2026-08-03T13:36:25+03:00 nezha systemd[1]: Stopping session-4.scope - Ses  
2026-08-03T13:36:25+03:00 nezha systemd[1]: Stopping session-42.scope - Se  
... (orderly stop of every user service: bluetooth, gdm, gnome-remote-desk  
    polkit, power-profiles-daemon, upower, mullvad-daemon, sshd, smb, team  
2026-08-03T13:36:25+03:00 nezha systemd[1]: Started plymouth-poweroff.serv  
2026-08-03T13:36:27+03:00 nezha systemd[1526]: Reached target gnome-sessio  
2026-08-03T13:36:27+03:00 nezha helix-nats[3506]: [1] ... [INF] Initiating  
2026-08-03T13:36:27+03:00 nezha helix-redis[3533]: 1:signal-handler (17857  
2026-08-03T13:36:27+03:00 nezha helix-redis[3533]: 1:M 03 Aug 2026 10:36:2  
2026-08-03T13:36:29+03:00 nezha systemd[1526]: Reached target shutdown.tar
```

```
2026-08-03T13:36:29+03:00 nezha systemd[1526]: Reached target exit.target
2026-08-03T13:36:30+03:00 nezha systemd[1]: Stopped systemd-logind.service
2026-08-03T13:36:31+03:00 nezha systemd[1]: Reached target umount.target -
```

Decisive evidence: **helix-redis[3533]: ... * User requested shutdown...** â€” Redisâ€™s own internal log line, written when it receives a clean SHUTDOWN/SIGTERM from an orderly system shutdown sequence, explicitly identifies this as a *user-requested* shutdown, not a crash, not a signal from an automated/scripted source, and not an idle-triggered action. This is corroborated by the surrounding sequence: every GNOME session service, every container (libpod-* scopes), NATS and Redis all received a clean, sequential SIGTERM-and-wait shutdown â€” the exact behavior of a deliberate poweroff/shutdown/GUI â€œPower Offâ€” action, never the behavior of a crash, panic, forced power-loss, or automated idle action.

None of the CONST-033 hardening was bypassed or defeated by this event. At the time of the event: sleep/suspend/hibernate targets were already masked, AllowSuspend=no was already set, IdleAction=ignore and HandlePowerKey=ignore were already configured (fix marker dated 2026-04-28, well before 2026-08-03). None of those settings prevent a human operator from directly invoking poweroff / shutdown -h now / systemctl poweroff / the desktopâ€™s â€œPower Offâ€” menu action â€” CONST-033â€™s hard ban targets *automated*, *agent-driven*, or *idle-triggered* power-state transitions; it does not, and was never intended to, prevent the human operator from manually powering off or restarting their own machine.

Conclusion

Verdict: a genuine, one-time, human-initiated shutdown+reboot occurred on 2026-08-03 13:36:25â€”13:37:26 MSK, five days before this triage. It is real (not a false positive of the screen-lock/compositor-stall/container-OOM classes), but it is **not a CONST-033 violation**: it was not a suspend, not triggered by idle timeout, not triggered by any automation/script/AI agent in this project or any other, and none of the source-treeâ€™s forbidden-command guardrails were bypassed. helix-redisâ€™s own log explicitly attributes it to a user-requested shutdown, and the entire teardown sequence is the signature of a deliberate, orderly power-off â€” never a crash or an automated trigger.

- â€” No suspend/hibernate ever invoked (0 matches, full window since fix).
- â€” No OOM-kill anywhere in the window (0 matches).
- â€” No forbidden host-power-management call exists anywhere in the actual (non-documentation) source tree.
- â€” The single flagged poweroff broadcast is fully explained: a real, orderly, human-requested shutdown+reboot, five days prior to this triage, unrelated to any automation.
- â€” CONST-033 hardening (masked targets, AllowSuspend=no, IdleAction=ignore, HandlePowerKey=ignore) was already in effect at

the time and remains intact; zero events of any kind since the resulting boot (4+ days continuous uptime at time of triage).

- `host_no_auto_poweroff_challenge.sh` test 4/4 will continue to FAIL going forward, because its “since fix” window is fixed at the 2026-04-28 fix-marker mtime and this is a real, permanently-recorded journal entry that predates the triage “this is expected, dated, stale signal from a real-but-benign one-time event, not a recurring or ongoing issue. No config change is warranted by this finding alone (the flagged event was legitimate operator action, not a hardening gap).
- Separate, pre-existing, unrelated finding:
`no_suspend_calls_challenge.sh` reports 3 false-positive hits inside `constitution/docs/scripts/guard-forbidden-commands.md` (documentation quoting the guard hook’s own denylist).
Recommended fix-forward (not applied here, per the read-only mandate of this triage): add that path to `EXCLUDE_PATHS` in `scripts/host-power-management/check-no-suspend-calls.sh`, alongside the other already-exempted governance docs.

Scope note

No power-related configuration, service, or system setting was modified as part of this triage. Every command run was read-only (`uptime`, `journalctl` queries, `journalctl --list-boots`, and the two pre-existing, source-verified-read-only challenge scripts). Nothing was committed as part of this triage; this document is left for review per instructions.